2.6 Data Abstraction in Structure

List allow data abstraction in that you can give a name to a set of memory cells. For instance, in the game Mastermind, you must keep track of a list of four colors that the player guesses. Instead of using four separate variables (color1, color2, color3, and color4) you can use a single variable 'Predict', e.g.,

Predict:=['red', 'blue', 'green', 'green']

What lists do not allow us to do is name the various parts of a multi- item object. In the case of a Predict, you don't really need to name the parts:

using an index to get to each color suffices.

But in the case of something more complex, like a person, we have a multi- item object where each 'item' is a named thing: the firstName, the lastName, the id, and the email. One could use a list to represent a person:

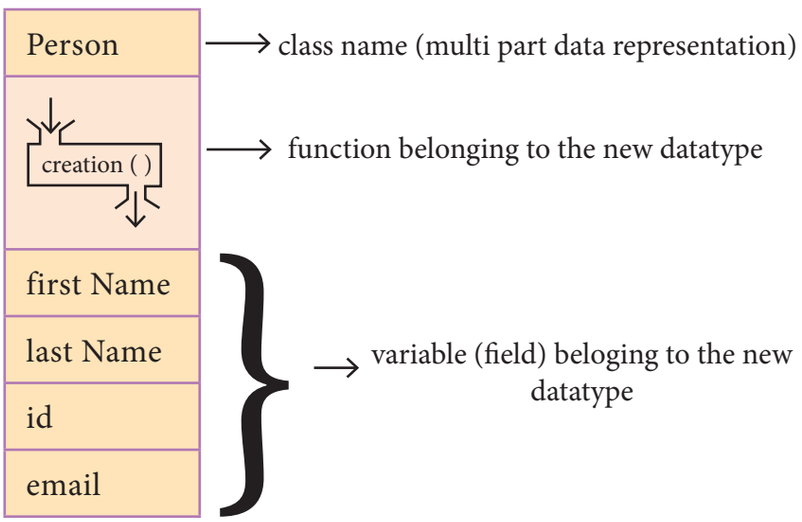
person:=['Padmashri', 'Baskar', '994-222-1234', 'compsci@gmail.com']

but such a representation doesn't explicitly specify what each part represents.

For this problem instead of using a list, you can use the structure construct (In OOP languages it's called class construct) to represent multi-part objects where each part is named (given a name). Consider the following pseudo code:

```
class Person:
    creation( )
    firstName := " "
    lastName := " "
    id := " "
    email := " "
```

The new data type Person is pictorially represented as



12th Computer Science_EM Chapter 2.indd 16 26-12-2022 17:25:50